LAB 1

DevOps Foundations & Continuous Integration

Task 3 – Automate Application Deployment

Student	Kunal
Platform	Ubuntu Linux + Jenkins + GitHub
Jenkins Job	Kunal-Lab1-Basic-CICD
Source Repository	kunalprajapati14042005/lab1-devops-version-control
Deployment Target	/tmp/lab1-deployed
Application	Static HTML web application

This report documents the actual Jenkins-based application deployment performed for Lab 1, including the configured build trigger, source checkout, Build/Test/Deploy pipeline, post-build action, and verification of the deployed application.

Submission evidence is based only on the screenshots captured during the completed experiment.

1. Objective & Deployment Workflow

The objective of Task 3 is to automate application deployment using Jenkins by configuring a build trigger, deployment steps, and a post-build action. The completed workflow uses GitHub as the source repository and Jenkins as the CI/CD automation server.

Implemented workflow

Requirement-to-evidence mapping

Requirement	Evidence captured
Build trigger	Jenkins Configure → Triggers → Poll SCM → H/5 * * * *
Deployment steps	Deploy stage removes/creates /tmp/lab1-deployed and copies app/index.html
Post-build action	Jenkins Declarative Post Actions reports deployment completion
Automation result	Jenkins Build #4 completes the pipeline successfully
Deployment verification	Deployed HTML is served locally and the browser displays the application

Key configuration

- Jenkins job: **Kunal-Lab1-Basic-CICD**
- Repository branch: **main**
- Trigger schedule: **H/5 * * * ***
- Deployment directory: **/tmp/lab1-deployed**
- Verification server: Python HTTP server on local port 8000

2. Build Trigger Configuration

Jenkins was configured with the Poll SCM trigger using the schedule H/5 * * * *. This instructs Jenkins to periodically check the configured source repository for changes and start a build when changes are detected.

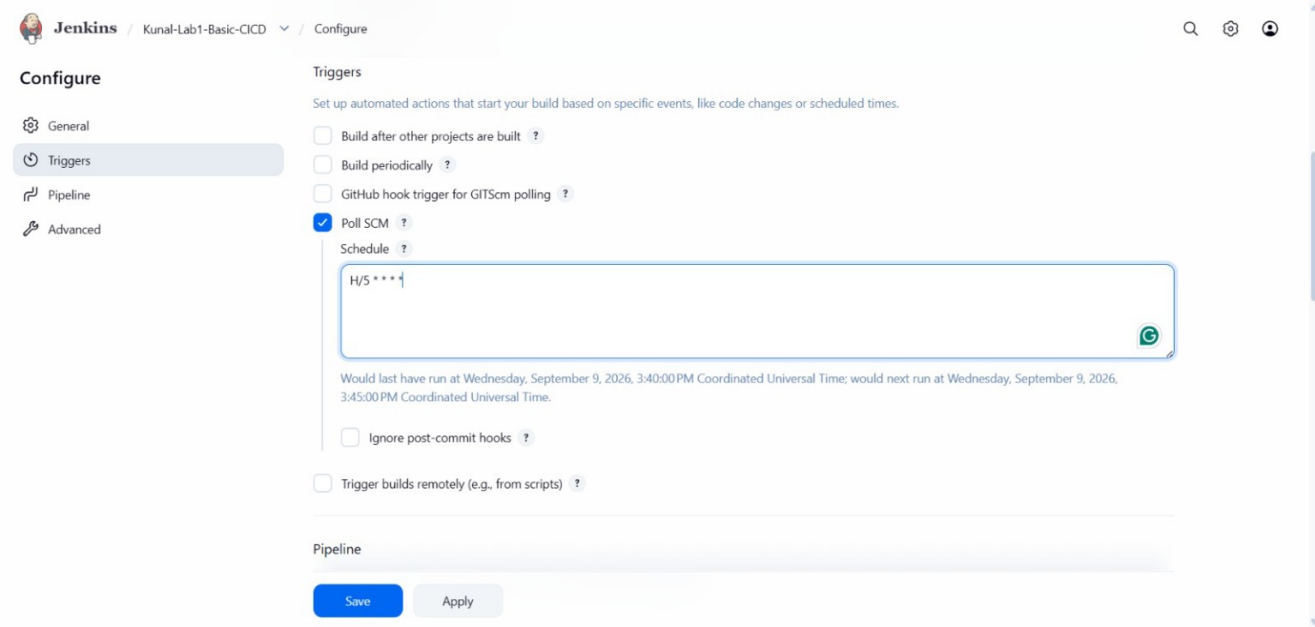

*Figure 1. Jenkins Trigger configuration showing Poll SCM enabled with schedule H/5 * * * *.*

Evidence interpretation

The screenshot directly shows the **Poll SCM** checkbox enabled and the configured schedule. This satisfies the build-trigger portion of Task 3.

3. Source Checkout & Pipeline Start

The Jenkins job obtains the Jenkinsfile from the GitHub repository and checks out the main branch before executing the pipeline. The console output identifies the repository and the revision used for Build #4.

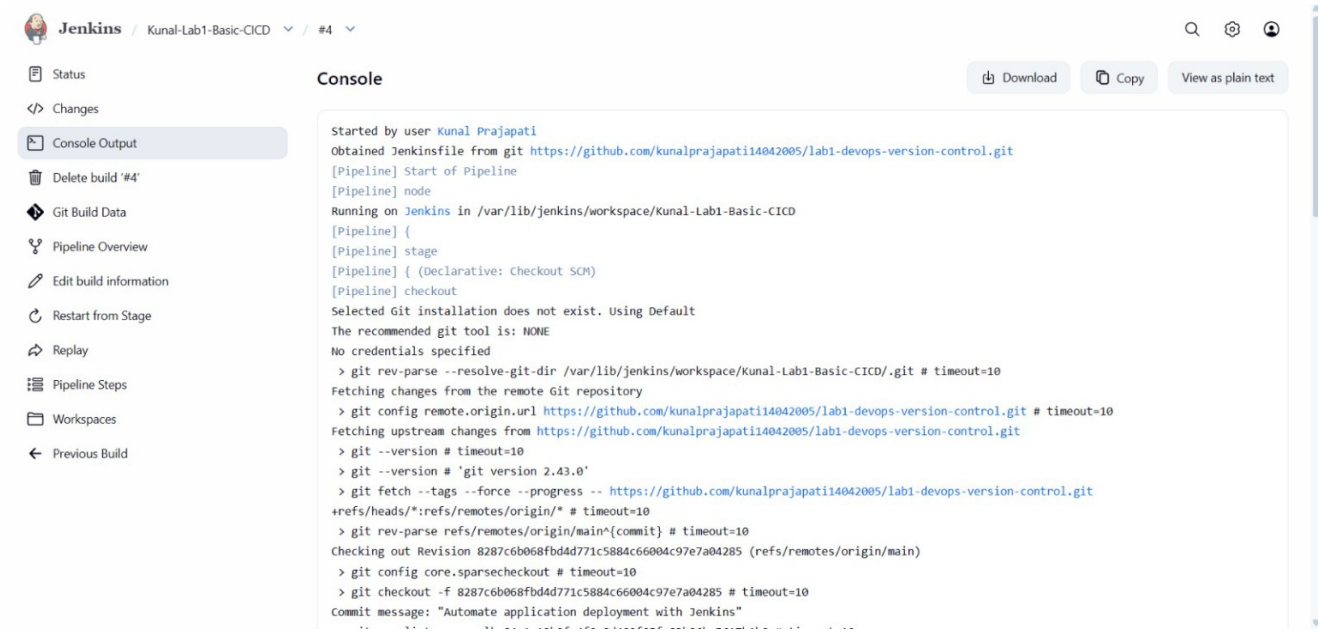

Figure 2. Jenkins Build #4 console showing GitHub repository checkout and pipeline initialization.

Observation

The source-controlled Jenkinsfile is retrieved from GitHub, allowing the deployment workflow to be executed automatically from the repository definition.

4. Build & Test Stages

After checkout, Jenkins executes the Build and Test stages. The Build stage verifies that app/index.html exists. The Test stage checks that the expected deployment text is present in the HTML file.

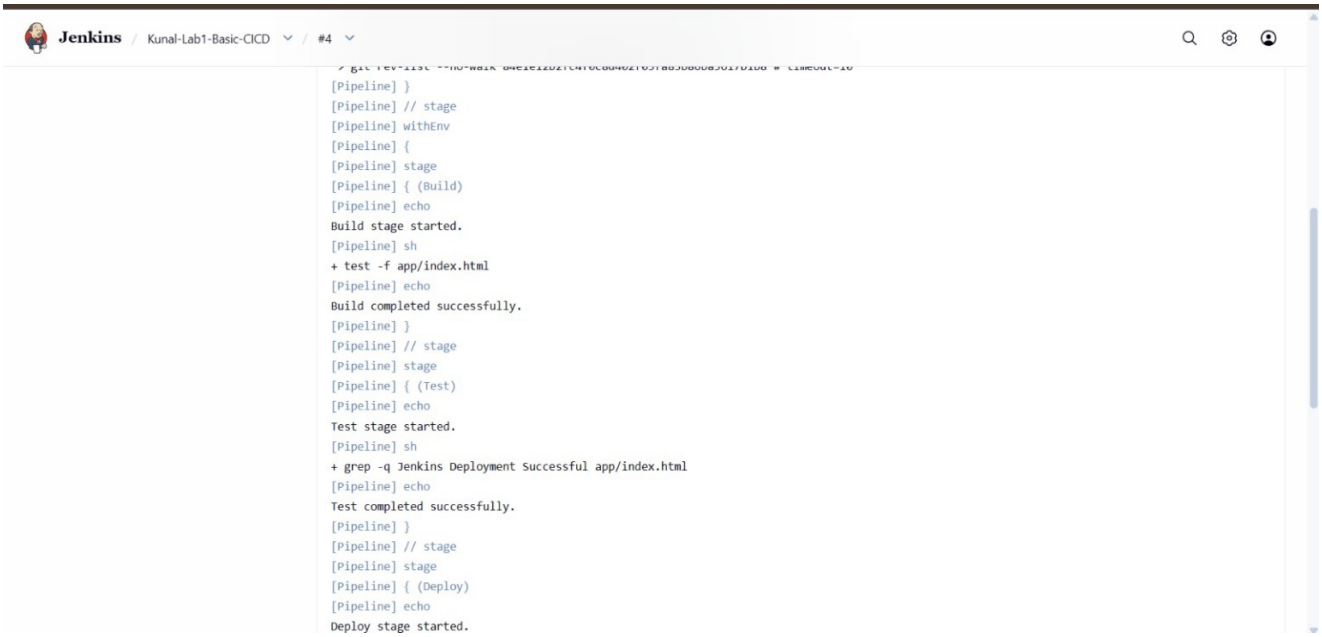

Figure 3. Jenkins Build #4 console showing successful Build and Test stage execution.

Result

Both stages complete without failure, establishing that the application artifact is present and passes the defined content check before deployment.

5. Deployment Steps & Post-build Action

The Deploy stage performs the actual file deployment. Jenkins removes any previous deployment directory, creates /tmp/lab1-deployed, copies the application files into it, and lists the resulting deployment contents.

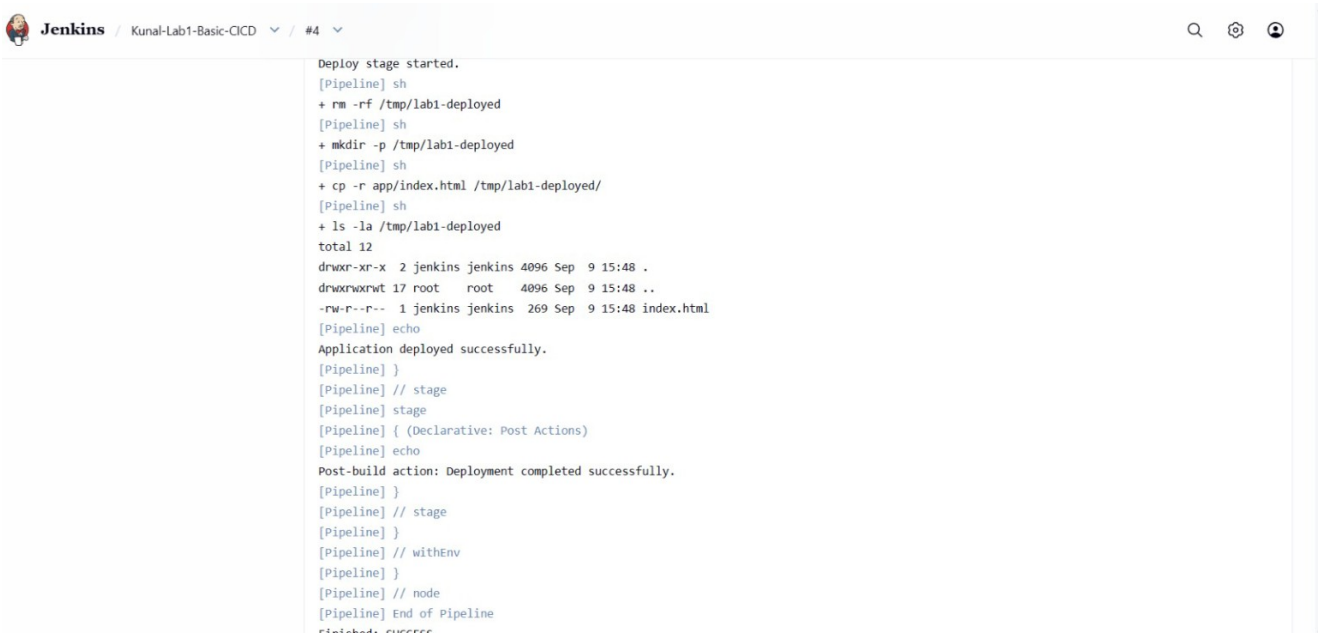

Figure 4. Jenkins Build #4 console showing deployment to /tmp/lab1-deployed and the successful post-build action.

Deployment evidence

The console shows **index.html** present in /tmp/lab1-deployed and reports **Application deployed successfully.** The Declarative Post Actions section then reports **Post-build action: Deployment completed successfully.**

6. Deployment Verification & Conclusion

The deployed files were served from /tmp/lab1-deployed using Python's local HTTP server. A curl request returned the deployed HTML content, and the browser displayed the deployed application page.

Command-line verification

```
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ cd /tmp/lab1-deployed
kunal_prajapati@LAPTOP-45509TK7:/tmp/lab1-deployed$ python3 -m http.server 8000 --bind 0.0.0.0 >/tmp/lab1-server.log 2>&
1 &
[1] 4025
kunal_prajapati@LAPTOP-45509TK7:/tmp/lab1-deployed$ echo $!
4025
kunal_prajapati@LAPTOP-45509TK7:/tmp/lab1-deployed$ curl http://localhost:8000
<!DOCTYPE html>
<html>
<head>
    <title>Lab 1 Jenkins Deployment</title>
</head>
<body>
    <h1>Jenkins Deployment Successful</h1>
    <p>Lab 1 - DevOps Foundations & Continuous Integration</p>
    <p>Application deployed automatically by Jenkins.</p>
</body>
</html>
kunal_prajapati@LAPTOP-45509TK7:/tmp/lab1-deployed$
```

Figure 5. Terminal verification showing the local HTTP server and the deployed HTML returned by curl.

Browser verification

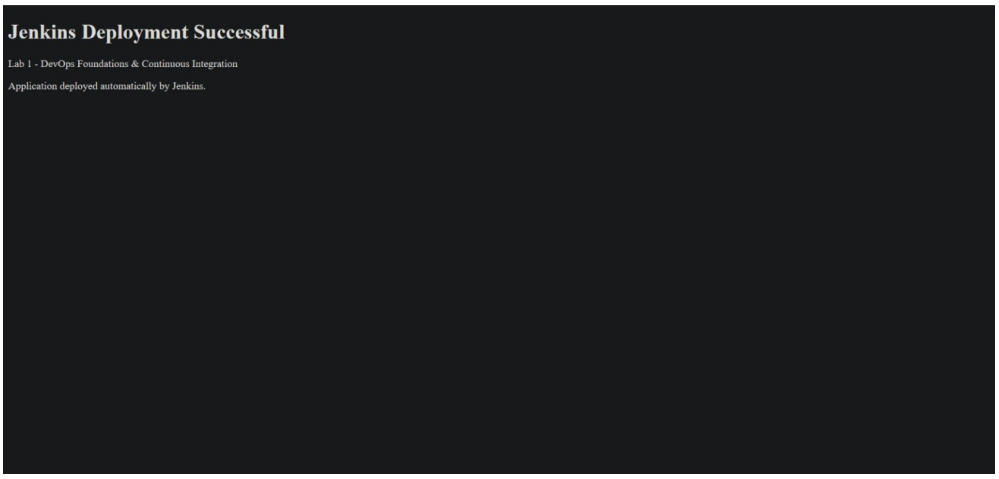

Figure 6. Browser displaying the deployed application with the message "Jenkins Deployment Successful".

Final conclusion

Task 3 was implemented using Jenkins automation. The workflow includes a configured Poll SCM build trigger, source checkout from GitHub, Build and Test stages, an actual deployment step to /tmp/lab1-deployed, a post-build action, and successful local application verification. The captured evidence demonstrates the complete deployment flow.